

DxHash: A Scalable Consistent Hashing Based on the Pseudo-Random Sequence

Chaos Dong, Fang Wang, Dan Feng
Huazhong University of Science and Technology
Email: chaosdong@hust.edu.cn

Abstract

Consistent hashing (CH) has been pivotal as a data router and load balancer in diverse fields, including distributed databases, cloud infrastructure, and peer-to-peer networks. However, existing CH algorithms often fall short in simultaneously meeting various critical requirements, such as load balance, minimal disruption, statelessness, high lookup rate, small memory footprint, and low update overhead. To address these limitations, we introduce DxHash, a scalable consistent hashing algorithm based on pseudo-random sequences. To adjust workloads on heterogeneous nodes and enhance flexibility, we propose weighted DxHash. Through comprehensive evaluations, DxHash demonstrates substantial improvements across all six requirements compared to state-of-the-art alternatives. Notably, even when confronted with a 50% failure ratio in a cluster of one million nodes, DxHash maintains remarkable processing capabilities, handling up to 13.3 million queries per second.

1 INTRODUCTION

When the concept of Consistent Hashing (CH) was first proposed [6], it rapidly gained popularity and assumed an essential role as a data router and load balancer in various fields, including distributed databases [13], [8], [2], cloud infrastructure [10], [18], and peer-to-peer networks [15], [5]. In these scenarios, CH evenly distributes loads (with keys) to buckets (or nodes) while maintaining consistency. Unlike conventional hashing functions, CH adheres to four notions of consistency: *Balance*, *Monotonicity*, *Spread*, and *Load* [6]. This paper assumes that node states are centrally managed by a controller, and each node holds the complete latest view of node states [11]. Consequently, the notions of *Spread* and *Load*, which come into effect in distributed hashing tables, aren't discussed in this paper. Similar to some previous work [9], as long as a hashing algorithm satisfies the properties of *Balance* and *Monotonicity*, it is considered consistent. Consistency is crucial to balancing load and protecting against large-scale data migration caused by topological changes in the cluster.

As CH finds wide application, in addition to the requirement of consistency, there are more pressing demands that need to be met. Recently, six properties have been identified to evaluate whether a CH algorithm is ideal:

- **Balance:** Ensures that each active node in the cluster has an equal probability of being mapped to by a key, resulting in a uniform load distribution among nodes.
- **Monotonicity** (or **Minimal Disruption**[3]): Guarantees that when a node is inserted or removed, the affected objects are remapped either from this node to other nodes or from other nodes to this node. This property avoids unnecessary remapping among unchanged nodes.
- **Statelessness:** A desirable property where the results of a hashing algorithm are unaffected by historical operations (e.g., queries, node additions, and node deletions). Statelessness reduces the overhead of serializing the operating order in distributed environments.
- **Lookup throughput:** Measures the rate at which objects are mapped to nodes, serving as a core performance indicator for CH algorithms.
- **Update overhead:** Refers to the overhead incurred when node states change, such as node failures/recoveries or cluster scaling/shrinking. An ideal CH algorithm should quickly update the mapping results in response to state changes to maintain efficient routing.
- **Memory footprint:** Considers the amount of memory consumed by the CH algorithm. A smaller memory footprint is desirable as it reduces costs and avoids performance degradation caused by excessive memory usage [10].

Many previously proposed methods have struggled to overcome the trade-off among the six requirements mentioned above. The original Consistent Hashing (CH), named Karger Ring or Ring in this paper, achieves minimal disruption but cannot simultaneously support both uniform balance and a low memory footprint [6]. MaglevHash [3], proposed by Google in 2014, faces a similar situation, although its lookup performance exceeds that of Karger Ring. SACH, proposed in 2021, is a CH algorithm similar to Maglev. Despite a series of optimizations, the memory footprint and update performance are still not ideal. Jump Consistent Hash [7] and AnchorHash [9] show relatively ideal performance in six properties, but the scalability of

TABLE 1: The Comparison of DxHash and Common CH Algorithms

	Ring[6]	Maglev[3]	JCH[7]	SACH[10]	AnchorHash[9]	DxHash
Balance	+	+++	+++	+++	+++	+++
Monotonicity	+++	+	+++	++	+++	+++
Statelessness	✓	✓ ^①	×	×	×	✓
Lookup	$O(\log(cn))$ ^②	$O(1)$	$O(\log(n))$	$O(1), O(\log(m))$ ^⑤	$O((1 + \ln(\frac{n}{a}))^2)$ ^⑥	$O(\frac{n}{a})$
Update	$O(\log(cn))$	$O(m \log(m))$ ^③	$O(1)$ ^④	$O(1), O(m)$	$O(1)$ ^⑦	$O(\frac{n-a}{n})$ or $O(1)$ ^⑧
Memory (Bytes)	$24cn$	$4m$	$O(1)$	$4m$	$16n$	$\frac{n}{8}$ to $5n$

^① Maglev and DxHash are statelessness for removals. The removal order of nodes does not affects the lookup results of these two CH algorithms.

^② *Karger Ring* introduces virtual nodes for balance. Constant c in *Karger Ring* denotes the number of the virtual nodes pointing to each physical node, and n is the number of physical nodes.

^③ m in Maglev is the size of the lookup table. **The value of m is recommended to be a prime number greater than $100n$** for the balance and minimal disruption.

^④ The updates in JCH are limited because only the last inserted node is allowed to be removed.

^⑤ The updates in SACH are limited because the total number of nodes cannot exceed the initial maximum size. Similar to Maglev, m in SACH denotes the size of the lookup table which is much larger than a . There are two update complexity matching the two update schemes in SACH.

^⑥ n is the upper bound of the cluster size, and a is the number of active nodes.

^⑦ The upper bound of AnchorHash is immutable, while DxHash supports *Scale* operation to double the upper bound.

^⑧ The update complexity and memory footprint of DxHash are determined by the detailed implementation.

these two has respective limitations.

In this paper, we propose DxHash, a scalable consistent hashing algorithm based on the pseudo-random sequence. By iteratively selecting possible nodes using a pseudo-random generator, DxHash provides nearly ideal performance that satisfies the mentioned six properties. In the evaluation, when the cluster size exceeds 1 million nodes and 50% of the nodes fail, DxHash can still process 13.3 million queries per second. Compared to state-of-the-art works, DxHash exhibits better lookup and update performance and improved scalability, with a smaller memory footprint. Furthermore, we combine distributed storage scenarios with DxHash to propose weighted DxHash. weighted DxHash adjusts the load on arbitrary nodes to make full use of hardware resources.

The rest of the paper is organized as follows. Section 2 introduces related works and motivation, comparing classical or state-of-the-art CH algorithms. Sections 3 and 4 introduce DxHash. Section 5 introduces weighted DxHash. In Section 6, we evaluate the performance of DxHash in comparison with existing CH algorithms. Finally, Section 7 concludes the paper. Here is a special thanks to chatGPT for contributions to improving the writing of this paper.

2 BACKGROUND

2.1 Related Work

Karger Ring, the original CH scheme proposed in 1997 [6], maps both nodes and keys into a cyclic hash space. The ring's values increase from 0 to 2^{32} in the clockwise direction, and each node is responsible for the keys in its assigned segment. While Karger Ring achieves *minimal disruption*, ensuring only the affected keys in the segment are remapped during node addition or removal, it struggles to maintain balance due to variable segment lengths. To address this, virtual nodes are introduced, raising the memory footprint [17]. Attempts to redistribute data for load balancing [4] introduce extra data migration and break *minimal disruption*. Moreover, the $O(\log(n))$ complexity of Karger Ring for both update and lookup raises concerns about performance.

Another CH algorithm, Highest Random Weight (HRW) [16], ensures complete balance and monotonicity. HRW assigns a unique identifier to each node and calculates random weights for mapping new items based on a combination of the item's key and the node's ID. The node with the highest weight is selected as the mapping result. However, HRW suffers from poor scalability due to significant computational overhead, resulting in a $O(n)$ lookup complexity, making it unsuitable for large-scale clusters.

MaglevHash, proposed by Google in 2016 [3], is a high-efficiency CH that maintains large memory tables, where keys are hashed to table indexes, and table contents are node IDs, allowing for $O(1)$ complexity queries. However, for balance, the table size is much larger than the number of nodes, introducing significant extra memory consumption. Additionally, MaglevHash struggles with *minimal disruption* and *low update complexity*.

Jump Consistent Hash (JCH) is a notable CH algorithm that leverages Pseudo-Random Sequence (PRS) [7]. JCH calculates a pseudo-random sequence based on a key and compares it with a specified probability to determine the node to which the key belongs. Although JCH meets standard CH requirements, it does not support arbitrary node additions or removals. Changes are restricted to the tail node, or else it would disrupt the *minimal disruption* property. Consequently, JCH is not suitable for scenarios involving random and frequent node updates.

Recent CH proposals, such as SACH [10] and AnchorHash [9], bring new perspectives. SACH uses double hashing similar to Maglev, with two update algorithms, fast but unbalanced, and slow but balanced. However, SACH still faces challenges in memory footprint and update complexity, and data skew increases with failure rates. AnchorHash, while near-ideal with $O(1 + \log(\frac{a}{w}))$ lookup complexity, faces issues with a fixed upper bound on the cluster size and strict statefulness, preventing concurrent updates.

2.2 Motivation

Since the existing CH algorithms have their own problems, we proposed DxHash, a stateless, scalable and consistent hash which meets the six requirements almost-perfectly. Table 1 shows the theoretical performance comparisons between DxHash and other CH algorithms. We can find that all aspects of DxHash are as good as or better than others, except that the lookup complexity is a bit worse than AnchorHash. However, the experimental results in Section 6.7 will show that the practical performance of DxHash is usually higher than AnchorHash due to a minor constant term in DxHash's lookup complexity. Besides, DxHash outperforms AnchorHash because: (1) DxHash supports the scaling/shrinking operations to adjust the upper bound of the number of nodes, (2) DxHash has stronger statelessness because its mapping results are independent of the removal order of nodes, and (3) the memory footprint of DxHash is only 0.8%-31% of that of AnchorHash.

3 DXHASH ALGORITHM

DxHash utilizes an array, called NSArray, to represent the state of nodes in a cluster or network. The size of the array is set to the smallest power of 2 greater than the number of active nodes in the cluster. For example, if the cluster consists of 4 nodes, the corresponding NSArray has a length of 8 (i.e., 2^3). Each active physical node is associated with an active item in the array. Figure 1a illustrates an example where the cluster contains 4 active nodes, labeled 0, 1, 2, and 3. As shown in the figure, the array items 0, 1, 2, and 3 are active (depicted in white), while the remaining items in the array are reserved for future node insertions and are referred to as inactive node IDs. In the given example, the inactive items 4, 5, 6, and 7 are unused and can be assigned to subsequently inserted nodes.

3.1 Lookup.

DxHash maps objects to nodes based on a Pseudo-Random Number Generator (PRNG) [1]. A PRNG is an algorithm to generate an infinite sequence composed of pseudo-random numbers within a given range. The generated sequence is not truly random, because it is determined by an initial value called the *random seed*. The PRNG features *randomness* and *reproducibility*. First, the generated numbers are distributed uniformly and randomly across the range. Second, the generated sequence can be reproduced if the seed is identical.

By using the digitized key (e.g., the key's hash value) as the *seed*, a PRNG produces a unique and reproducible sequence of random numbers for each key. By taking the items of the generated sequence modulo the size of the NSArray, DxHash obtains a sequence of node IDs that includes both active and inactive node IDs. This node-ID sequence, denoted as S , is used in determining the location of the key. We use $S[i]$ to denote the i th item in the sequence. The key's location is determined by selecting the first active item $S[i]$. Figure 1a illustrates an example of querying keys $k1$ and $k2$ in a 4-node cluster. For $k1$, the generated node-ID sequence S is 1, 4, 5, 2, Since the first item in the sequence, 1, corresponds to an active node, $k1$ is mapped to

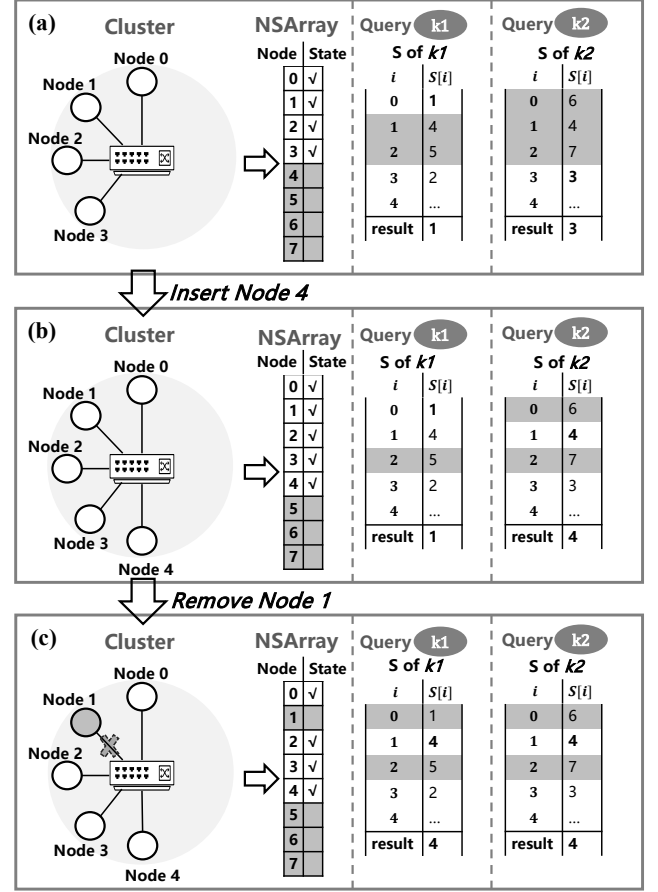


Fig. 1: An example of how DxHash works for a cluster. (a) Query $k1$ and $k2$. (b) Insert node 4. (c) Remove node 1.

node 1. For $k2$, S is 6, 4, 7, 3, ..., and the first active item in the sequence is $S[3] = 3$, resulting in $k2$ being mapped to node 3.

It is important to note that the node-ID sequence S is calculated in real-time upon each query and is not pre-stored in memory. The number of calculations required to find an active node is referred to as the search length. When a user queries a key, they first calculate $S[0]$. If the result corresponds to an active node ID, it is the final result of the query, and the search length is 1, as shown in the example of querying $k1$ in Figure 1a. If the result is an inactive node ID, the user continues to calculate the subsequent node IDs ($S[1], S[2], \dots$) until an active node ID is found, incrementing the search length accordingly. Hence, in Figure 1a, the search length for $k2$ is 4.

3.2 Update.

As mentioned previously, the size of the NSArray is typically larger than the cluster size to accommodate future node insertions. When a new node joins the cluster, it is assigned an inactive node ID. In Figure 1b, we illustrate an example where a new node joins the cluster and is assigned the inactive node ID 4. The corresponding item in the NSArray is then updated to active. As a result, the first active node ID in the generated node-ID sequence S for $k2$ becomes 4, leading to the remapping of $k2$ to node 4.

DxHash removes nodes by marking the corresponding items in the NSArray as inactive, thus preparing them for future assignments. Figure 1c demonstrates the removal of node 1. The item 1 in the NSArray is set to inactive, causing a change in the mapping result of $k1$. Initially, $k1$ was mapped to node 1, but since node 1 is now inactive, $k1$ is remapped to node 4.

3.3 Proof of Minimal Disruption and Balance.

In this section, we provide a proof for the properties of Minimal Disruption and Balance guaranteed by DxHash.

Theorem 1 (Minimal Disruption): DxHash ensures Minimal Disruption.

Proof: Let k be an arbitrary key and S be the generated node-ID sequence. We assume that the n th entry in S ($S[n]$) represents the original mapping result of k . To prove Minimal Disruption, we consider the cases of node removal and addition:

(i) *Removal:* Suppose node b is being removed. If $S[n] = b$, it implies that k was originally mapped to the removed node b . The change in mapping does not violate Minimal Disruption. If $S[n] \neq b$, it means that k was not initially mapped to node b . Since $\forall m < n$, the state of node $S[m]$ remains inactive and is unaffected by the removal of a node, the key k continues to be mapped to node $S[n]$.

(ii) *Addition:* Let b be a newly added node. If $\exists l < n$ such that $b = S[l]$, we define the minimum value of l as l_{min} . As $l_{min} < n$, for $\forall m < l_{min}$, node $S[m]$ is inactive. Consequently, k will be remapped to node $S[l_{min}] = b$ because it becomes the first active node in S . If $\forall l < n, b \neq S[l]$, the addition of node b does not impact the mapping of k , and therefore, the mapping of k remains unchanged.

Thus, we can see that the changed node is either the original or the destination of the remapped keys. Consequently, DxHash achieves Minimal Disruption.

Theorem 2 (Balance): In DxHash, there is an equal probability for a key to be mapped to each active node.

Proof: The process of locating a key across nodes involves repeated calculations to (pseudo-)randomly generate node IDs. DxHash terminates the calculations only when a generated node ID corresponds to an active node. At the i th round, the probability distribution of $S[i]$ among all active nodes is uniform due to the randomness of the PRNG. Since the probability distribution at each calculation round is uniform, the overall distribution is also uniform. Thus, the Balance property is proven.

3.4 Boundary cases

We have demonstrated that DxHash guarantees *Minimal Disruption* and *Balance*. However, there are certain boundary cases that need to be considered. In theory, with an ideal PRNG, DxHash can always access an active node as long as there exist active nodes in the cluster. However, in practice, the PRNG is not truly random and may fail to reach any active nodes. Additionally, excessively long search lengths are not acceptable. The uncontrollable search length is a unique problem for DxHash that other algorithms do not have. To bridge the gap between theory and practice, we introduce a threshold for the search length. In our implementation, the threshold is set to $8n$, where n denotes the cluster size.

Although setting this threshold affects balance and minimal disruption, we believe it is necessary to avoid the mentioned boundary cases. Moreover, even in scenarios where there is only one active node in a large cluster, the number of affected keys whose search length exceeds the threshold is tiny. The probability of a key not matching the only active node after $8n$ searches is:

$$\mathbb{P} = \left(\frac{n-1}{n} \right)^{8n} \quad (1)$$

When n is sufficiently large, $\mathbb{P}$ is approximately $\frac{1}{e^8}$, indicating that only 0.03% of keys are affected by the threshold. This calculation demonstrates that terminating the search whose length is larger than $8n$ has no noticeable impact in practice.

One another boundary case is that, the cluster requires to scale when the cluster is full of active nodes. Some CH algorithms [10], [9] do not support *Scale* operations for two reasons. First, scaling out a cluster may require a complete remapping of keys, which can be a resource-intensive task. Second, this situation can be avoided by setting a large enough initial upper bound for the cluster. In contrast, DxHash supports *Scale* operations to cater to broader applications. In DxHash, the cluster size is limited by the size of the NSArray. When the cluster reaches its maximum capacity and all items in the NSArray are active, DxHash behaves as a classic hash algorithm that maps objects to nodes with a single calculation. To scale out the cluster, DxHash doubles the size of the NSArray and sets the new items to inactive. This doubling of the range in the classic hash algorithm results in only half of the loads needing to be migrated. As a result, DxHash reduces the remapping effort by half. It's important to note that remapping half the loads can still be a significant task. Therefore, the *Scale* operation is suitable for inserting active nodes in batches to amortize the overhead. For scenarios involving only a few node updates, it is recommended to initialize the NSArray with a sufficiently large size.

Figure 2 illustrates an example of a *Scale* operation. Initially, there are 3 active nodes, and the length of the NSArray is four. After inserting a new node, the NSArray becomes full, and inserting another node would require a complete remapping. To avoid this, the length of the NSArray is expanded to 8. Items 1-4 in the array are active, while the remaining items are marked as inactive, ready for subsequent node insertions.

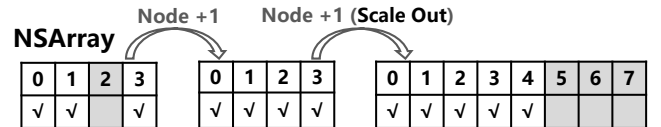


Fig. 2: An example of the scaling operation

4 DxHASH IMPLEMENTATION

This section presents the implementation details of native DxHash, a preliminary version of DxHash. Native DxHash utilizes two data structures: NSArray and IQueue. The

NSArray is implemented as a boolean array, with each item occupying only 1 bit. This bit represents the state of the corresponding node, indicating whether it is active (1) or inactive (0). On the other hand, DxHash employs a 4-byte (32-bit) integer queue called IQueue to store inactive node IDs. The functionality of IQueue will be further discussed in §4.2.

Native DxHash supports four essential functions: *Lookup*, *AddNode*, *RmNode*, and *Init*. Algorithm 1 outlines the functions.

Algorithm 1: Native DxHash

```

/* This function receives a given key
   and output the corresponding node.
   */
1 Function Lookup( $k$ ):
   Result: A working node ID ( $n_{ID}$ )
2    $r \leftarrow k$ ;
3   repeat
4      $r \leftarrow R(r)$ ;
5      $n_{ID} \leftarrow r \bmod |\text{NSArray}|$ ;
6   until  $\text{NSArray}[n_{ID}] = 1$ ;
7   return  $n_{ID}$ ;

/* This function finds an inactive node
   ID and rests the ID to active.      */
8 Function AddNode():
   Result: An inactive node ID ( $n_{ID}$ )
9    $n_{ID} \leftarrow \text{IQueue.pop}()$ ;
10   $\text{NSArray}[n_{ID}] \leftarrow 1$ ;
11  return  $n_{ID}$ ;

/* This function receives an active
   node ID to remove it.              */
12 Function RmNode( $n_{ID}$ ):
   Result: Void
13   $\text{NSArray}[n_{ID}] \leftarrow 0$ ;
14   $\text{IQueue.push}(n_{ID})$ ;
15  return;

/* This function initializes the IQueue
   by a given NSArray.                */
16 Function Init( $\text{NSArray}$ ):
   Result: Void
17   $\text{IQueue} \leftarrow \emptyset$ ;
18  for  $item \in \text{NSArray}$  do
19    if  $item = 0$  then
20       $\text{IQueue.push}(item)$ ;
21  return;

```

4.1 Lookup

Lookup is the core function of DxHash, responsible for mapping a key to its corresponding node ID. DxHash utilizes a PRNG $R(x)$ to generate pseudo-random numbers cyclically (lines 3-6), with the key serving as the random seed (line 2). By performing modulo operation with the size of the NSArray, a random node ID is obtained (line 5). The loop continues until an active node is encountered (line 6), at which point the loop terminates, and the node ID is returned as the lookup result (line 7).

The time complexity of the *Lookup* function significantly impacts DxHash's performance. Let n denote the length of the NSArray and a denote the number of active nodes. We use p to represent the fraction $\frac{a}{n}$, which corresponds to the active ratio in the NSArray. In each iteration of the *Lookup* function (lines 3-6), the probability of hitting an active node is p , while the probability of hitting an inactive node is $(1 - p)$. The distribution of hitting an active node at each iteration follows the Bernoulli Distribution, and the number of iterations (i.e., search length) follows the Geometric Distribution [12]. Denoting the search length as τ , the expected value of τ is:

$$\mathbb{E}(\tau) = \frac{1}{p} \quad (2)$$

Substituting $p = \frac{a}{n}$ back into Formula 2, we find:

Theorem 3 (Query Complexity): In DxHash, given the size of the NSArray, n , and the number of active nodes, a , the Average Search Length (ASL) for keys is $\frac{n}{a}$.

4.2 AddNode

The *AddNode* function is responsible for node insertions in DxHash. When a new node joins the cluster, DxHash assigns an inactive ID to the node and adjusts the corresponding item in the NSArray. The key issue in this process is obtaining an inactive ID efficiently. Instead of performing a linear search on the NSArray, which has a time complexity of $O(n)$, DxHash introduces a new data structure called IQueue to expedite insertions. IQueue is a 4-byte (32-bit) integer queue that stores all inactive node IDs for fast insertions. In the pseudocode of the native DxHash (Alg. 1), lines 9-11 demonstrate how DxHash obtains an inactive node ID from the IQueue in constant time complexity ($O(1)$).

4.3 RmNode

The *RmNode* function handles node removals. When a node is removed, DxHash updates the data structures, namely the NSArray and IQueue. In Alg. 1, the *RmNode* function receives an active node ID as input. First, the corresponding item in the NSArray is set to 0 to indicate that the node is inactive. Then, the node ID is added to the IQueue for future assignments. The time complexity of the *RmNode* function is also $O(1)$.

4.4 Scale

DxHash supports the scale and shrink operations to adjust the upper bound of the cluster size. When the NSArray is full and new nodes are ready to join, the *Scale* operation is triggered. Alg. 2 presents the pseudocode for the *Scale* function. In this operation, the size of the NSArray is doubled (lines 2-3), and the new node IDs are set to be inactive (lines 4-5).

4.5 Shrink

The *Shrink* function is used when there are too many inactive nodes in the cluster. First, DxHash counts the number of active nodes with IDs greater than $\frac{|\text{NSArray}|}{2}$ (lines 8-12, Alg. 2). Then, the size of the NSArray is halved (line 13), and

the IQueue is rebuilt based on the updated NSArray (line 14). Finally, DxHash reassigns the same number of nodes counted in the first step (lines 15-16).

When scaling out or shrinking the cluster, DxHash operates on the NSArray and IQueue, involving a maximum of n nodes. The time complexity of both the *Scale* and *Shrink* operations is $O(n)$. However, the number of remapped keys caused by the *Shrink* operation is relatively greater than that caused by the *Scale* operation. This is because the *Shrink* operation includes additional node deletions and insertions. The *Shrink* operation is triggered only when the active ratio is very small (e.g., 1%).

Algorithm 2: Scale adjustment

```

1 Function Scale_out():
   Result: The cluster size after adjustment
2    $n \leftarrow |\text{NSArray}|$ ;
3   NSArray is resized to  $2n$ ;
4   for  $i \in [n, 2n)$  do
5      $\text{RmNode}(i)$ ;
6   return  $2n$ ;
7 Function Shrink():
   Result: The cluster size after adjustment
8    $n \leftarrow |\text{NSArray}|$ ;
9    $\text{count} \leftarrow 0$ ;
10  for  $i \in [\frac{n}{2}, n)$  do
11    if  $\text{NSArray}[i] = 1$  then
12       $\text{count} \leftarrow \text{count} + 1$ ;
13  NSArray is resized to  $\frac{n}{2}$ ;
14   $\text{Init}(\text{NSArray})$ ;
15  for  $i \in [0, \text{count})$  do
16     $\text{AddNode}()$ ;
17  return  $\frac{n}{2}$ ;

```

4.6 Optional Trade-off

4.6.1 New method for insertion: minimal memory footprint but slower insertions.

DxHash introduces the IQueue data structure to accelerate insertions, but this comes at the cost of storage efficiency and statelessness. The IQueue can vary in length from 0 to n , and each item in the queue requires 4 bytes of storage. On average, the expected memory footprint of the IQueue is $2n$ bytes. In comparison, the size of the NSArray is much smaller, only $\frac{n}{8}$ bytes. Additionally, the use of IQueue for insertions introduces a dependency on the removal order, which compromises the statelessness property. The sequence of node IDs in the IQueue is determined by the order of removals. Consequently, the order of insertions becomes deterministic and stateful, which goes against the desired statelessness characteristic of the DxHash algorithm. To address this, we propose an alternative insertion approach for DxHash to achieve less memory and stronger statelessness without the need for an additional data structure.

The key to inserting a node is to obtain an inactive node ID for assignment. The new insertion approach is inspired by the *Lookup* procedure in DxHash, which accesses

NSArray items pseudo-randomly and repeatedly until an inactive item is found. The pseudocode for this approach is shown in Alg. 3. In lines 2-6, we reuse the code from the *Lookup* procedure with two modifications. First, we change the random seed to use a constant instead of a given key for reproducibility, which is 1228 shown as line 2. Second, we terminate the loop when an inactive node is selected instead of an active one, as we are looking for an inactive node ID (line 6). Similar to the calculation in Formula 2, the time complexity of this insertion approach can be estimated as $O(\frac{n}{n-a})$.

Algorithm 3: AddNode_2

```

/* This function finds an inactive node
   ID and resets the ID to active. */
1 Function AddNode():
   Result: An inactive node ID ( $n_{ID}$ )
2    $r \leftarrow 1228$ ;
3   repeat
4      $r \leftarrow R(r)$ ;
5      $n_{ID} \leftarrow r \bmod |\text{NSArray}|$ ;
6   until  $\text{NSArray}[n_{ID}] = 0$ ;
7    $\text{NSArray}[n_{ID}] \leftarrow 1$ ;
8   return  $n_{ID}$ ;

```

This new insertion approach strikes a balance between space footprint, statelessness, and update efficiency. Firstly, since no additional data structure is required, the memory footprint of DxHash is solely determined by the size of the NSArray, which is $\frac{n}{8}$ bytes or 125 KB per million nodes. This memory footprint is only 0.8% of that of *AnchorHash* ($16n$ bytes). Secondly, the new design achieves stronger statelessness, as the insertion order is unaffected by the history of node removals. The shortcoming is the higher insertion complexity, which is $O(\frac{n}{n-a})$ now. Although the insertion efficiency decreases, it is still superior to most CH algorithms. The size of the NSArray n is twice the number of active nodes a initially. When there are only a few additions or removals, the time complexity of $O(\frac{n}{n-a})$ remains relatively constant and independent of the absolute cluster size. In the worst-case scenario where only one inactive node exists ($n - a = 1$), the time complexity is $O(n)$, which is no worse than the method of linear search. It is worth noting that the cases where no inactive or active nodes exist are not considered in this section, as the boundary cases are discussed separately in §3.4. In summary, DxHash without IQueue is a stateless CH algorithm with a small memory footprint and acceptable update overhead.

4.6.2 NSArray organized in Bytes: faster lookup but larger memory.

The native implementation of DxHash uses 1 bit for each node to significantly reduce the memory footprint. However, since memory devices are typically byte-addressed, performing operations on bytes is much faster than on individual bits. Therefore, when high lookup performance is desired, the NSArray in DxHash can be organized as a byte array, with each node's state represented by a single byte.

Compared to the native implementation of DxHash, this solution sacrifices some storage efficiency in exchange for improved lookup performance. However, the memory footprint is still lower than that of *AnchorHash*. As the NSArray becomes a byte array, its memory footprint is equal to n bytes. Taking into account the memory usage of IQueue, which is $4a$ at most, the total memory requirement for this approach is $5n$ bytes, resulting in a 69% reduction in memory compared to *AnchorHash* ($16n$ bytes).

5 WEIGHTED DXHASH

Weighted DxHash is introduced to address the issue of load distribution in clusters or networks consisting of heterogeneous physical nodes. Conventional consistent hashing schemes use virtual nodes to adjust load distribution, where multiple virtual nodes point to the same physical node, effectively multiplying the load on that node. However, virtual nodes lead to increased memory footprint and cannot accurately distribute the load. In response, we propose Weighted DxHash.

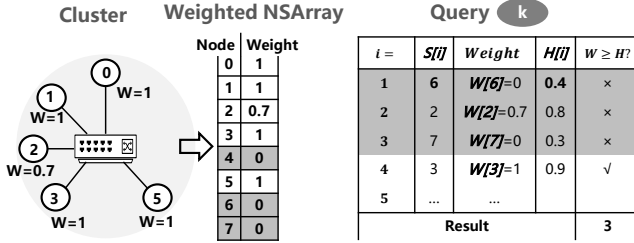


Fig. 3: An example of querying a key in a 5-node weighted cluster via weighted DxHash. The mapping result of the key is node 3.

Weighted DxHash extends the native DxHash by introducing node weights and another PRNG called H . Each node is assigned a random floating-point weight between 0 and 1. Nodes with higher weights can handle more loads, while the weight of an inactive node is set to 0. The PRNG H is used to generate a pseudo-random floating-point sequence within the range of $[0, 1]$, with $H[i]$ representing the i th item in the sequence.

Figure 3 illustrates an example of querying a key using Weighted DxHash. The cluster shown in the figure consists of 5 nodes, with node 2 having a weight of 0.7 and all other nodes having a weight of 1. The cluster is represented as a weighted NSArray of length 8. The weights of the inactive items in the array (nodes 4, 6, 7) are set to 0. The lookup process in Weighted DxHash is completed in two steps.

In step 1, similar to DxHash, Weighted DxHash generates a random node ID $S[i]$ at each calculation cycle. In the right part of Figure 3, the random node IDs generated for the key k are 6, 2, 7, 3, ... in sequence.

In step 2, Weighted DxHash generates $H[i]$ and compares it with the weight of node $S[i]$. If the weight is no less than $H[i]$, Weighted DxHash terminates the loop and returns the current node ID as the mapping result. Otherwise, Weighted DxHash proceeds to the next cycle for further searching. From the last column in Figure 3, we observe that in the first 3 cycles, the weights of the generated

node IDs are always smaller than $H[i]$. However, at $i = 4$, the weight of node $S[4] = 3$ is 1, which is larger than $H[4] = 0.9$. Therefore, node 3 becomes the final mapping result for the key k .

The main idea behind Weighted DxHash is to influence the probability of a key being mapped to a node based on the node's weight. In step 1 of Weighted DxHash, keys pointing to node $S[i]$ with weight W have a probability of W to be accepted by that node in step 2. As the weight decreases, the node becomes less likely to accept keys, resulting in a reduced load. When the weight is 0, the node rejects all keys, effectively having no load. If weights are only set to 0 and 1, Weighted DxHash is downgraded to native DxHash.

The time complexity of the *Lookup* operation in Weighted DxHash can be analyzed as follows. The probability for a key to hit a node in round i is $\frac{\sum_{x=0}^{n-1} W_x}{n}$, where n is the size of the weighted NSArray and W_x is the weight of node x . There are two calculations performed at each round, one for S and another for H . The number of calculations required to query a key, denoted as τ , follows the Geometric Distribution, and its expectation is given by:

$$\mathbb{E}(\tau) = \frac{2n}{\sum_{i=0}^{n-1} W_i} \quad (3)$$

The expectation of the load on node b , denoted as l_b , can be calculated using the following formula:

$$\mathbb{E}(l_b) = \frac{W_b}{\sum_{i=0}^{n-1} W_i} * L \quad (4)$$

where L represents the total load across all nodes.

In terms of space complexity, the weighted NSArray in Weighted DxHash is organized as a 4-Byte (32-bit) floating-point array. The rest of the implementation remains the same. Therefore, the expected memory footprint is $8n$ Bytes, which includes both the 4-Byte NSArray and the 4-Byte IQueue.

Weighted DxHash overcomes the limitations of virtual nodes and provides flexibility in load distribution. By combining virtual nodes with node weights, the load distribution can be adjusted according to the performance of individual nodes. Nodes with lower performance can be assigned smaller weights to reduce their load, while high-performance nodes can have multiple virtual nodes to fully utilize their capabilities.

6 EVALUATION

In this section, we compare and evaluate the performance of different consistent hashing (CH) algorithms, including *Karger Ring* (Ring), *MaglevHash* (Maglev), *AnchorHash* (AH), and DxHash (DH). We also consider different trade-offs in the implementation of DxHash, such as using a 1-bit NSArray or a 1-Byte NSArray, and using IQueue or not. These different implementations are denoted as DH-b, DH-B, and DH-IQ, respectively. Since deploying a large-scale cluster for testing is challenging, we evaluate the CH algorithms through local simulations.

To evaluate the performance of DxHash, we initialize an NSArray with 1 million entries, representing the states of 1

million mock nodes (active or inactive). We then generate batches of 32-bit integers as keys and feed them to DxHash. The algorithm returns the corresponding mock node IDs. We measure the lookup performance as the rate of successful key queries, the memory footprint as the amount of memory used by the process, and the update overhead as the time consumed to adjust the data structures when inserting new nodes. Similar evaluation methods are used for the other CH algorithms.

6.1 Environment

As shown in Table 2, all experiments are performed on the same commercial machine with the processor of Intel Xeon CPU E5-2620 0 @ 2.00GHz and 32 GB memory. The system is CentOS 7.8. The kernel version is 3.10.0-1127, and the GCC version is 7.3.1. All algorithms are implemented in C++. The PRNG to generate S is implemented as a hardware-supported CRC32 [14] hash function, which is a uniform hash function for 32-bit integers with high randomness and speed in number generation.

TABLE 2: Environment Configuration

Processor	Intel Xeon CPU E5-2620 0 @ 2.00GHz
Memory	32GB
Operating System	CentOS Linux release 7.8.2003 (Core)
Kernel Version	3.10.0-1127.13.1.el7.x86_64
GCC Version	7.3.1 20180303

In this section, we evaluate DxHash (DH) in terms of memory footprint, lookup throughput, update overhead, load balancing, minimal disruption, fault tolerance, and elasticity.

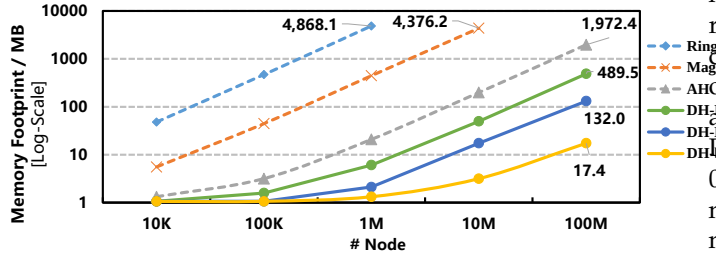


Fig. 4: Memory Footprint of Ring, Maglev, AH, DH-B&IQ, DH-B, DH-b in the cluster whose size varies from 10K to 100M.

6.2 Memory Footprint

CH algorithms have varying memory footprints depending on their implementation. Ring and Maglev allocate additional memory space for load balancing or to minimize migration after node updates. Ring uses a Red-Black Tree (RB-Tree) implementation, with each node occupying 24 Bytes of memory. In our experiments, we create 100 virtual nodes for each physical node in Ring to achieve load balance. Therefore, Ring's memory footprint is approximately $2.4n$ KB. Maglev utilizes a large lookup table for key routing, with each entry requiring 4 Bytes of memory. We allocate 100 times the minimum required memory for Maglev to ensure less than 1% imbalance [3], resulting in a memory

footprint of $400n$ Bytes. AH has a memory footprint of $16n$ Bytes, while DH's memory footprint varies depending on the version. We implement three versions of DH. DH-b, which uses a bit array as NSArray, requires $\frac{n}{8}$ Bytes of memory. DH-B, which uses a byte array as NSArray, occupies n Bytes. DH-B&IQ, based on DH-B, utilizes a 32-bit IQueue to store inactive node IDs, resulting in a memory footprint of up to $5n$ Bytes. Theoretical analysis confirms that DH has a smaller memory footprint compared to other CH algorithms.

To validate the theoretical analysis, we collected the memory footprints of the CH algorithms for different numbers of nodes and present the results in Figure 4. The x-axis represents the number of nodes ranging from 10K to 100M. The experimental results align with our theoretical derivation. In a 100-million-node cluster, AH requires 2 GB of memory, DH-B&IQ occupies 500 MB, DH-B requires 100 MB, and DH-b only uses 17 MB. DH-b is the most memory-saving implementation, reducing the memory footprint by 98.4% compared to AH. DH-B&IQ uses most memory in the three DH's implementations, still with a reduction on memory footprint of 75.2%.

6.3 Lookup Throughput

We compare the lookup throughput of Ring, Maglev, AH, and DH. Since DH offers optional trade-offs between lookup efficiency and memory footprint, we test two versions of DH: DH-B, which uses a byte array as the NSArray for higher lookup rate, and DH-b, which uses a bit array for a smaller memory footprint.

Figure 5a shows the impact of cluster scale on the lookup performance of the different CH algorithms. We vary the number of nodes from 1K to 10M and measure the lookup rate in Million Keys Per Second (MKPS) on the y-axis. In this case, all nodes are active, resulting in an active ratio (a/n) of 1. Figures 5(b) and 5(c) demonstrate how the active ratio affects the lookup performance of AH and DH (including DH-b and DH-B). The active ratio is varied between 1, 0.9, 0.5, and 0.1. Figure 5b corresponds to a cluster with 1 million nodes, while Figure 5c represents a cluster with 10 million nodes. From the figures, we can make several observations:

① DH-B outperforms all other CH algorithms in terms of lookup throughput. AH performs slightly worse than DH-B, while Maglev has better lookup performance than Ring but is still inferior to DH-B and AH. This is because the lookup complexity of Ring is $O(\log(n))$, which is relatively worse than the $O(1)$ complexity of other CH algorithms.

② Maglev and AH exhibit satisfactory lookup performance when the number of nodes is small. For example, when there are 1,000 nodes, Maglev achieves a lookup rate of 41.5 MKPS, and AH achieves a lookup rate of 44.64 MKPS, which is close to the lookup rate of DH-B. However, their lookup performance decreases as the number of nodes increases. When there are 10 million nodes, Maglev's lookup rate drops to only 5.73 MKPS, and AH's lookup rate drops to 11.92 MKPS. This rapid decline is due to their excessive memory footprint, which becomes a bottleneck for quick querying as the number of nodes increases. In contrast, DH-B maintains a lookup rate of 31.06 MKPS even with a 10-million-node cluster, which is 2.6 times higher than

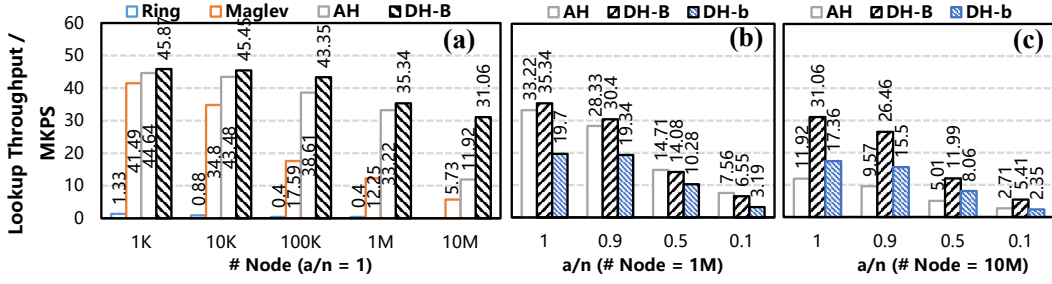


Fig. 5: The lookup rate when handling 100 million queries. (a) Lookup comparison of Maglev, Ring, AH and DH-B in clusters with 1K, 10K, 100K, 1M and 10M nodes. The active ratio of AH and DH-B (a/n) is 1. (b) Lookup comparison of AH, DH-B, and DH-b when the active ratio (a/n) varies from 1 to 0.1. The cluster size is 1 million. (c) Lookup comparison of AH, DH-B, and DH-b when the cluster size is 10 million.

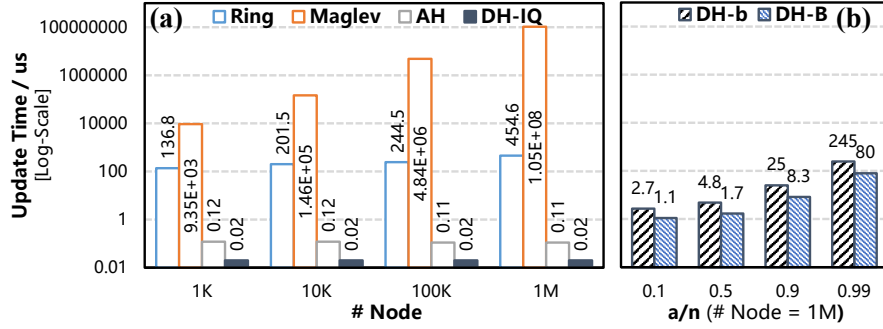


Fig. 6: Insertion overhead comparison. (a) Insertion latency of Maglev, Ring, AH and DH-IQ in clusters with 1K, 10K, 100K, and 1M nodes. (b) Insertion latency of DH-B and DH-b when the active ratio (a/n) varies from 0.1 to 0.99. The cluster size is 1 million.

AH and 5.4 times higher than Maglev. This high lookup throughput is due to DH's nearly constant complexity and its tiny memory footprint.

② Figure 5b shows the lookup performance of AH, DH-B, and DH-b in a 1-million-node cluster with different active ratios. As the active ratio decreases, the lookup throughput of all three algorithms decreases. However, AH experiences a slower drop in performance compared to DH-B. When the active ratio is less than 0.5, DH-B performs worse than AH. This is because the lookup complexity of DH has a linear correlation with the active ratio ($O(a/n)$), while AH has a logarithmic correlation with the active ratio ($O((1 + \ln(\frac{n}{a}))^2)$).

⑦ From Figure 5c, we observe that although AH has better lookup complexity, it performs much worse than DH-B in a 10-million-node cluster. The lookup throughput of DH-B is $2 - 2.7\times$ higher than that of AH. This can be attributed to DH's smaller memory footprint, which allows it to be stored in the CPU cache for faster access. Additionally, DH has a simpler lookup method that requires fewer memory accesses compared to AH, which is discussed in Section 6.7. The smaller and simpler data structure in DH incurs lower memory access costs compared to the larger and more complex data structure in AH.

Overall, DH-B demonstrates comparable lookup throughput with AH and outperforms other CH algorithms, particularly in large-scale clusters. Its high performance can be attributed to its nearly constant complexity, minimal memory footprint, and efficient lookup method.

6.4 Update overhead

Update overhead is another critical metric for evaluating CH algorithms. Figure 6 illustrates the time required for inserting nodes into different CH algorithms. DH is implemented in three versions based on optional trade-offs. DH-IQ uses the IQueue for fast insertions, while DH-B and DH-b employ a slower method that accesses NSArray repeatedly and randomly to select an inactive node ID for assignment. Although the latter method is slower, it reduces memory footprint significantly. The efficiency of this method is related to the active ratio, where smaller active ratios result in faster insertions. The difference between DH-B and DH-b lies in the data structure of NSArray, with DH-B using a byte array and DH-b using a bit array.

In Figure 6(a), we observe the influence of cluster scale on the update cost of CH algorithms. The x-axis represents the node number, ranging from 1K to 1M, while the y-axis represents the time taken to adjust the CH algorithm after inserting a new node. The results are averaged over 100 trials. Figure 6(b) displays the update cost of DH-B and DH-b for different active ratios. We make the following observations from the figures:

① Maglev exhibits the longest update time, taking more than 100 seconds to insert a node into a 1-million-node cluster. This high overhead is due to its high complexity. Each node insertion requires $O(m \log(m))$ time, where m is the size of the large lookup table, which is $100\times$ greater than the number of nodes n .

② Ring’s update time grows slowly as the cluster expands. When the number of nodes increases from one thousand to one million, the update time changes from 137 us to 455 us. The update overhead of Ring is moderate compared to other schemes, as its update complexity is $O(\log(n))$, which is far less than that of Maglev.

③ AH and DH-IQ exhibit very short update times, each less than 1 us. With a constant update complexity, they can update node states on a nanosecond scale, regardless of the cluster scale.

④ Compared to DH-IQ, DH-B and DH-b have higher insertion overhead. As the active ratio increases from 0.1 to 0.99, the update time of DH-b increases from 2.7 to 245 us, while that of DH-B increases from 1.1 to 80 us. DH-B performs better than DH-b because the byte array has higher efficiency for updates. However, both schemes perform worse than DH-IQ, indicating that the IQueue significantly reduces insertion overhead.

Overall, DH-IQ demonstrates the best update performance among the DH variants, while AH also performs impressively. Maglev exhibits the highest update time due to its complex update process, while Ring maintains a moderate update overhead.

6.5 Balance

We compare the load balancing performance of Ring, Maglev, AH, and DH. It is worth noting that we do not test DH in different versions because the mentioned optional trade-offs do not affect the load distributions. The cluster consists of 1000 active nodes. Ring’s load balancing depends on the design of virtual nodes. Hence, we evaluate Ring with each physical node matching 1, 10, 100, and 1000 virtual nodes, respectively. Maglev’s load balancing is influenced by the size of the lookup table, which we set as prime numbers approximately 10 times, 100 times, and 1000 times larger than the node size. Since AH and DH not use over-provisioned memory for balance, they are test once as shown in the right of Figure 7a. We randomly generate 100 million integers as keys to query the corresponding node IDs. The load on each node is measured by the number of keys assigned to it. In this case, the average load on each node is $100M/1000 = 100K$ queries. We quantify load balance using the standard deviation (σ). A smaller standard deviation indicates better load balance in a CH algorithm.

The experimental results are shown in Figure 7a. The x-axis represents the number of virtual nodes per physical node, which impacts the load balance of Ring and Maglev. AH and DH are evaluated in a cluster with 1000 active nodes. The y-axis represents the standard deviation of loads in the different CH algorithms. To facilitate comparison, all results are normalized to DH’s standard deviation. From the figure, we can observe that all algorithms, except for Ring, achieve good load balance. Ring exhibits the worst load balance, with a standard deviation approximately $7.8\times$ higher than that of DH. These results align with previous research findings [7].

6.6 Minimal Disruption

We evaluate the remapping ratio of Ring, Maglev, AH, and DH after node insertions. We gradually insert 100 nodes into

a 100-node cluster until the number of active nodes reaches 1000. The cluster size ranges from 100 to 1000 in increments of 100. As shown in Figure 7b, the ideal remapping ratio is calculated by dividing the number of updated nodes by the total number of nodes. The ideal remapping ratios after each insertion are $100/200 = 0.5$, $100/300 = 0.33$, $100/400 = 0.25$, and so on, represented by the bars in the figure. After each insertion, we provide duplicate sets of 100 million keys as input to the CH algorithms and calculate the corresponding node IDs. We compare the current results with the previous insertion’s results, count the number of remapped keys, and divide it by 10 million to obtain the real remapping ratio. The real remapping ratios of the different CH algorithms are represented by the dashed lines in Figure 7b. Comparing the real remapping ratios to the ideal remapping ratio, we observe that all schemes, except for Maglev, exhibit remapping ratios close to the ideal value. Maglev deviates slightly from the ideal remapping ratio, indicating that it cannot achieve complete minimal disruption, which is consistent with previous research [3].

6.7 Fault Tolerance

In this section, we test the fault tolerance of AH and DH, as they have demonstrated comparable performance in the previous experiments. DH and AH both require multiple searches to return mapping results, and the search length increases as the node active ratio decreases. We measure their fault tolerance using the average search length (ASL). Initially, the number of nodes is set to 1000, and we gradually remove 100 nodes until only 100 active nodes remain. This procedure reduces the active ratio from 1 to 0.05. Figure 8a illustrates the ASL as a function of the active ratio. DH exhibits a larger ASL than AH due to its higher lookup complexity ($O(\frac{n}{a})$) compared to AH’s complexity ($O(1 + \log(\frac{n}{a})^2)$) [9]. However, Section 6.3 demonstrated that the lookup throughput of DH is higher. This is because AH is a stateful algorithm that incurs more memory access overhead to maintain the update order of nodes. AH requires four memory accesses for each search, while DH only requires one. To compare the memory access overhead, we present AH’s Average Number of Memory Access (ANMA) for each lookup in Figure 8a, where $ANMA_{AH} = 4 \times ASL_{AH}$. The results show that AH consistently has a larger ANMA than DH until the active ratio drops below 7%. Therefore, when the active ratio is above 7%, DH consistently outperforms AH in terms of lookup performance, despite its higher lookup complexity.

6.8 Elasticity

DH supports the operations of scaling and shrinking to adjust the cluster size dynamically. In this section, we specifically evaluate the shrinking operation. We compare three schemes: AH, DH-B without shrink, and DH-B with shrink. The shrinking operation is triggered when the active ratio falls below 0.1. We measure the lookup throughput of the three schemes in a cluster with 1 million nodes and active ratios of 0.1, 0.01, and 0.001. The experimental results are shown in Figure 8b. From the figure, we make two observations.

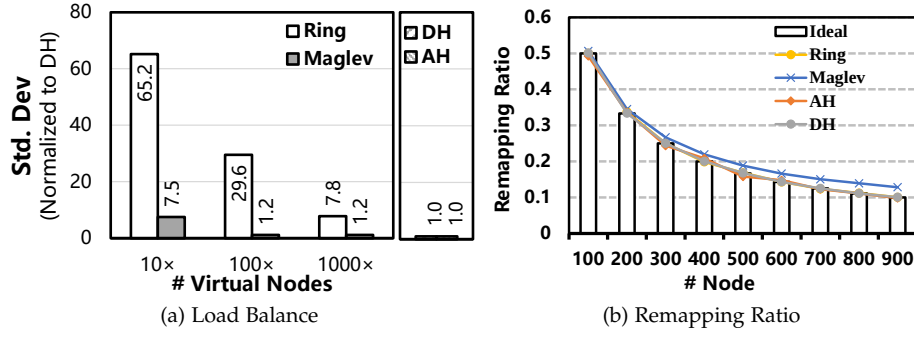


Fig. 7: (a) Load balance comparison when the number of virtual nodes per physical node varies from 10 to 1000. The Y-axis is the standard deviation normalized to DH. (b) The remapping ratio after inserting (9×100) nodes into a 100-node cluster. Bars represents the ideal remapping ratio, and the dash lines represents real remapping ratio of CH algorithms.

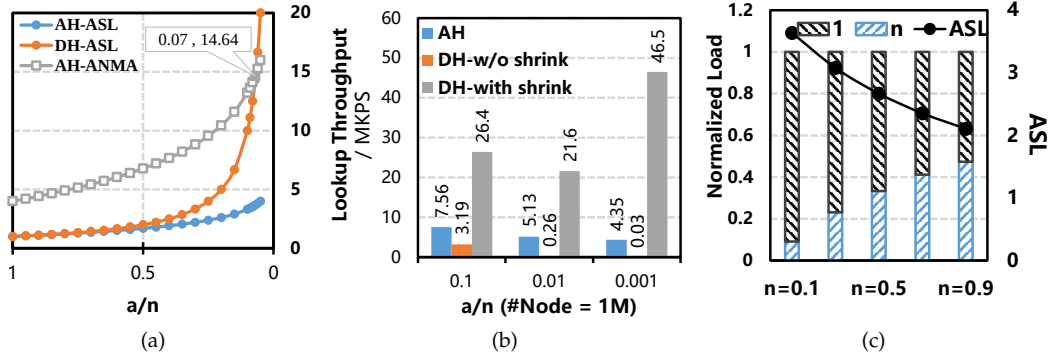


Fig. 8: (a) The Average Search Length (ASL) and Average Number of Memory Accesses (ANMA) of querying keys. The X-axis is the active ratio. (b) The lookup performance of AH, DH-B without shrinking, and DH-B with shrinking when the failure ratio is 0.9, 0.99, and 0.999 in 1 million nodes. (c) The ASL (drawn as lines) and load per node (drawn as stacked bars) in weighted DH. The cluster size is 1000, and the load is 100 million lookups. The weights of 512 nodes are 1, and the weights of others are n . Here, n is set to 0.1, 0.5, and 0.9 respectively.

First, the scheme of DH without shrink performs poorly when the active ratio is low. Compared to AH, which maintains a lookup rate of 4.35 MKPS even at an active ratio of 0.001, DH-B only achieves a lookup rate of 0.03 MKPS. This confirms that DH has a higher lookup complexity than AH. Second, the shrinking operation enhances the elasticity of DH. When shrinking is triggered, DH dynamically adjusts the upper bound of the cluster based on the number of active nodes. As a result, the active ratio increases, significantly improving the lookup throughput.

It is worth noting that while scaling and shrinking operations improve lookup performance, they come at the cost of remapping a large number of keys. The remapping ratio depends on the ratio of the cluster size before and after scaling or shrinking. For example, if a cluster shrinks to 1% of its original size, the remapping ratio is approximately 99%. If a cluster doubles its size, the remapping ratio is 50%. Considering the significant remapping overhead, scaling and shrinking operations are best suited for inserting or removing active nodes in batches to amortize the remapping cost.

6.9 Weighted DxHash

In this section, we evaluate weighted DxHash and verify whether the load distribution aligns with the theoretical derivation. We construct a weighted NSArray consisting of 1024 items, divided into two halves. The weights of one half are uniformly set to 1 (referred to as 1-nodes), while the weights of the other half range from 0.1 to 0.9 in steps of 0.2 (referred to as n -nodes). We generate 10 million random keys as input for weighted DxHash. The loads on each node are measured by the number of keys mapped to that node. Additionally, we record the average search length (ASL) for all keys. Figure 8c displays the normalized loads on the two parts and the average ASL. The ASL and the load distribution align with Formula 3 and 4 within an error margin of 0.1%. This confirms that weighted DxHash effectively adjusts the loads on nodes based on their weights, demonstrating quantitative load distribution control.

7 CONCLUSIONS

This paper introduces DxHash, an efficient, scalable, and adaptable consistent hashing algorithm. We present the algorithm, its implementation, and provide a complexity

proof for DxHash. Building on naive DxHash, we propose weighted DxHash. The evaluation of DxHash, compared to other existing CH algorithms, demonstrates its ability to maintain millions of nodes while delivering a high key lookup rate, occupying minimal memory footprint, and requiring minimal time for node additions or removals. Weighted DxHash also achieve its design objectives. Finally, the source code for DxHash and all associated tests are available as open source ¹.

REFERENCES

- [1] Elaine Barker, Elaine Barker, William Burr, William Polk, Miles Smid, et al. *Recommendation for key management: Part 1: General*. National Institute of Standards and Technology, Technology Administration . . . , 2006.
- [2] Chanwoo Chung, Jinhyung Koo, Junsu Im, Arvind, and Sungjin Lee. Lightstore: Software-defined network-attached key-value drives. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '19*, page 939–953, New York, NY, USA, 2019. Association for Computing Machinery.
- [3] Daniel E. Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilengiroglu, Bin Cheyney, Wentao Shang, and Jinnah Dylan Hosein. Maglev: A fast and reliable software network load balancer. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 523–535, Santa Clara, CA, March 2016. USENIX Association.
- [4] Xiang Fu, Can Peng, and Weihong Han. A consistent hashing based data redistribution algorithm. In Xiaofei He, Xinbo Gao, Yanning Zhang, Zhi-Hua Zhou, Zhi-Yong Liu, Baochuan Fu, Fuyuan Hu, and Zhancheng Zhang, editors, *Intelligence Science and Big Data Engineering. Big Data and Machine Learning Techniques*, pages 559–566, Cham, 2015. Springer International Publishing.
- [5] Pulkit Goel, Kumar Rishabh, and Vasudeva Varma. An alternate load distribution scheme in dhds. In *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 218–222, 2017.
- [6] David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web. In *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing, STOC '97*, page 654–663, New York, NY, USA, 1997. Association for Computing Machinery.
- [7] John Lamping and Eric Veach. A fast, minimal memory, consistent hash algorithm, 2014.
- [8] Zaoxing Liu, Zhihao Bai, Zhenming Liu, Xiaozhou Li, Changhoon Kim, Vladimir Braverman, Xin Jin, and Ion Stoica. Distcache: Provable load balancing for large-scale storage systems with distributed caching. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 143–157, Boston, MA, February 2019. USENIX Association.
- [9] Gal Mendelson, Shay Vargaftik, Katherine Barabash, Dean H. Lorenz, Isaac Keslassy, and Ariel Orda. Anchorhash: A scalable consistent hash. *IEEE/ACM Transactions on Networking*, 29(2):517–528, 2021.
- [10] Yuichi Nakatani. Structured allocation-based consistent hashing with improved balancing for cloud infrastructure. *IEEE Transactions on Parallel and Distributed Systems*, 32(9):2248–2261, 2021.
- [11] Vladimir Olteanu, Alexandru Agache, Andrei Voinescu, and Costin Raiciu. Stateless datacenter load-balancing with beamer. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 125–139, Renton, WA, April 2018. USENIX Association.
- [12] Andreas N Philippou, Costas Georgiou, and George N Philippou. A generalized geometric distribution and some of its properties. *Statistics & Probability Letters*, 1(4):171–175, 1983.
- [13] Jiwei Shu, Youmin Chen, Qing Wang, Bohong Zhu, Junru Li, and Youyou Lu. Th-dpms: Design and implementation of an rdma-enabled distributed persistent memory storage system. *ACM Trans. Storage*, 16(4), October 2020.
- [14] Ronak Singhal. Inside intel® core microarchitecture (nehalem). In *2008 IEEE Hot Chips 20 Symposium (HCS)*, pages 1–25. IEEE, 2008.
- [15] I. Stoica, R. Morris, D. Liben-Nowell, D.R. Karger, M.F. Kaashoek, F. Dabek, and H. Balakrishnan. Chord: a scalable peer-to-peer lookup protocol for internet applications. *IEEE/ACM Transactions on Networking*, 11(1):17–32, 2003.
- [16] D.G. Thaler and C.V. Ravishanker. Using name-based mappings to increase hit rates. *IEEE/ACM Transactions on Networking*, 6(1):1–14, 1998.
- [17] Xiaoming Wang and Dmitri Loguinov. Load-balancing performance of consistent hashing: Asymptotic analysis of random node join. *IEEE/ACM Transactions on Networking*, 15(4):892–905, 2007.
- [18] Chenggang Wu, Vikram Sreekanti, and Joseph M. Hellerstein. Autoscaling tiered cloud storage in anna. *Proc. VLDB Endow.*, 12(6):624–638, February 2019.

1. The code is available at <https://github.com/ChaosD/DxHash>